

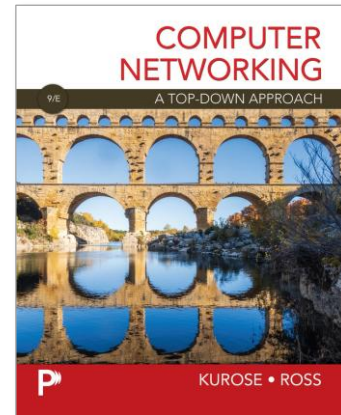
Wireshark Lab:

HTTP v9.0

Supplement to *Computer Networking: A Top-Down Approach*, 9th ed., J.F. Kurose and K.W. Ross

“Tell me and I forget. Show me and I remember. Involve me and I understand.” Chinese proverb

© 2005-2025, J.F Kurose and K.W. Ross, All Rights Reserved



We're now going to use Wireshark to investigate protocols in operation. In this lab, we'll explore several aspects of the HTTP protocol: the basic GET/response interaction, HTTP message formats, retrieving large HTML files, retrieving HTML files with embedded objects, and HTTP authentication and security.

As you answer each question, include a screenshot of the whole Wireshark window(s) with the supporting packet(s) selected and the necessary header(s) displayed. Mention in your answer the packet number(s) and the corresponding header(s) supporting your answer.

Before you run Wireshark, there are several things you should check in your network connection and browser configuration:

- Make sure you are **not** running a VPN (virtual private network) service. We'll learn about VPNs later in this course. But for now, all you need to know is that when you're running a VPN service, upper-layer protocol information (HTTP, TCP) that is sent by your computer may be encrypted. This makes it impossible to use Wireshark to look at what's going on inside these protocols!
- Make sure your browser program is **not**, by default, using the HTTP/3 protocol or the QUIC protocol. We'll learn about HTTP/3 and QUIC in Chapter 2. But for now, all you need to know is that when you're running HTTP/3 or QUIC, upperlayer protocol information (HTTP, TCP) sent by your computer will be encrypted. As of 2025, most popular browsers have adopted HTTP/3 and QUIC as defaults. A document that describes how to *disable* HTTP/3 AND QUIC as the defaults for your browser is here: <https://techysnoop.com/disable-quic-protocol-in-chromeedge-firefox/> .
- Make sure then when you go to a URL, that the URL starts with “http://” rather than with <https://> as the latter case will encrypt frames.

- Make sure privacy and browser privacy settings are turned off.
- Make sure you clear your browser cache and browser history before you start.

1. The Basic HTTP GET/response interaction

Let's begin our exploration of HTTP by downloading a very simple HTML file - one that is very short, and contains no embedded objects. Do the following:

1. Start up your web browser.
2. Start up the Wireshark packet sniffer, as described in the Introductory lab (but don't yet begin packet capture). Enter "http" (just the letters, not the quotation marks, and in lower case) in the display-filter-specification window, so that only captured HTTP messages will be displayed later in the packet-listing window. (We're only interested in the HTTP protocol here, and don't want to see the clutter of all captured packets).
3. Wait a bit more than one minute (we'll see why shortly), and then begin Wireshark packet capture.
4. Enter the following to your browser
<http://panday.upm.edu.ph/~bryann/cs138/wireshark-http/wireshark-httpsimple.html>
 Your browser should display the very simple, one-line HTML file.
5. Stop Wireshark packet capture.

Your Wireshark window should look similar to the window shown in Figure 1. If you're unable to run Wireshark on a live network connection, you can download a packet trace that was created when the steps above were followed.¹

¹ You can download the zip file <http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-9e.zip> and extract the trace file http-wireshark-trace1-1. These trace files can be used to answer these Wireshark lab questions without actually capturing packets on your own. Each trace was made using Wireshark running on one of the author's computers, while performing the steps indicated in the Wireshark lab. Once you've downloaded a trace file, you can load it into Wireshark and view the trace using the *File* pull down menu, choosing *Open*, and then selecting the trace file name. The resulting display should look similar to Figure 1 (for the http-wireshark-trace1-1 trace file for this HTTP lab). The Wireshark user interface displays just a bit differently on different operating systems, and in different versions of Wireshark.

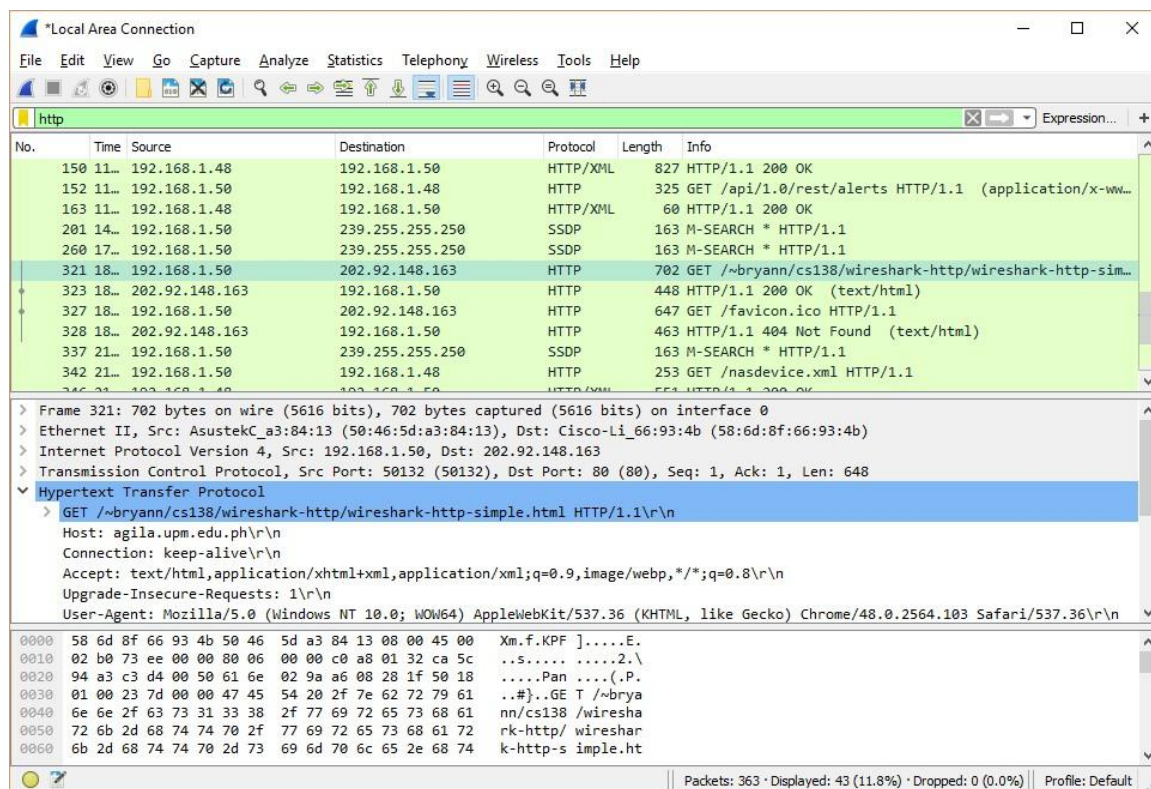


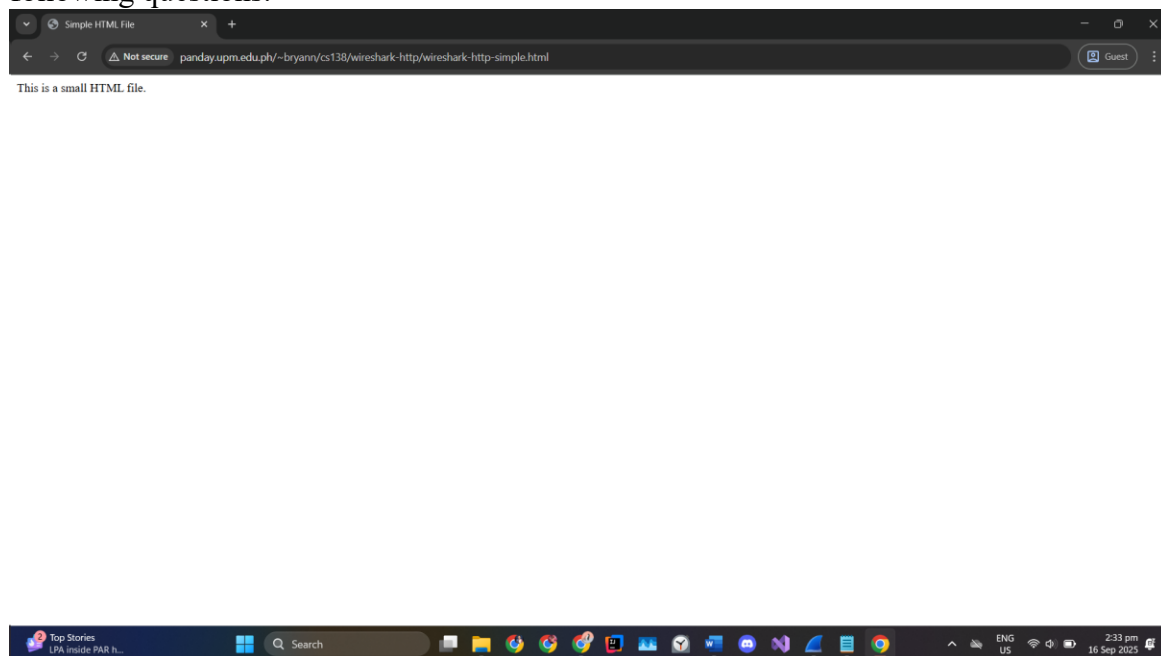
Figure 1: Wireshark Display after <http://panday.upm.edu.ph/~bryann/cs138/wiresharkhttp/wireshark-http-simple.html> has been retrieved by your browser

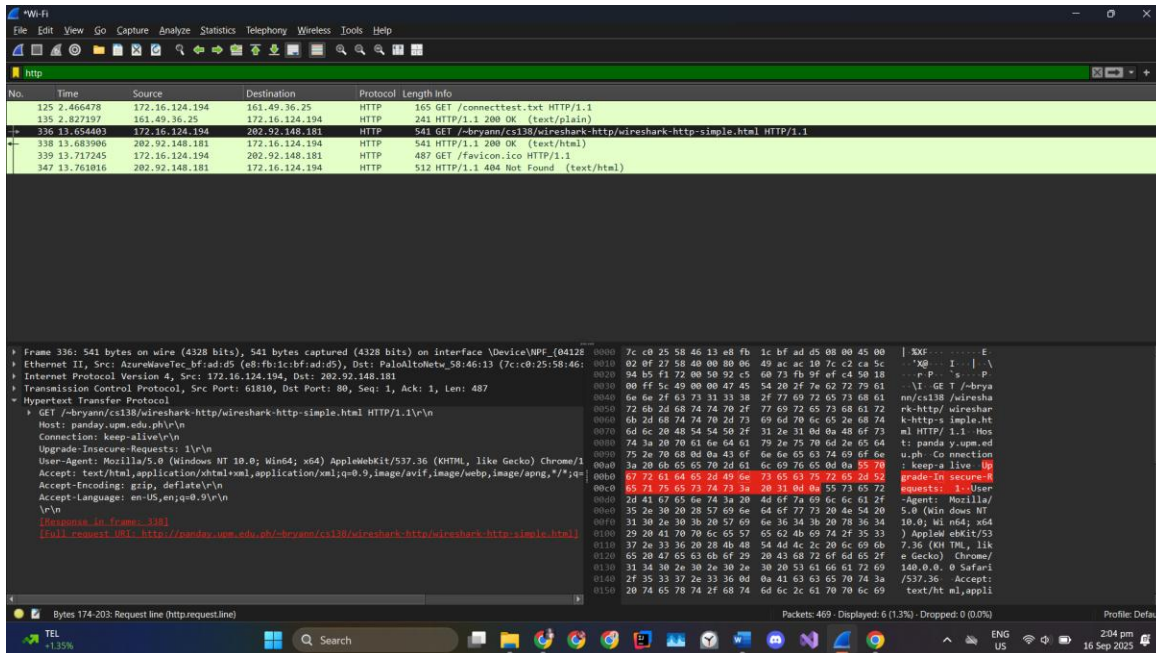
The example in Figure 1 shows in the packet-listing window that two HTTP messages of interest were captured: the GET message (from your browser to the panday.upm.edu.ph

web server) and the response message from the server to your browser. The packetcontents window shows details of the selected message (in this case the HTTP OK message, which is highlighted in the packet-listing window). Recall that since the HTTP message was carried inside a TCP segment, which was carried inside an IP datagram, which was carried within an Ethernet frame, Wireshark displays the Frame, Ethernet, IP, and TCP packet information as well. We want to minimize the amount of non-HTTP data displayed (we're interested in HTTP here, and will be investigating these other protocols in later labs), so make sure the boxes at the far left of the Frame, Ethernet, IP and TCP information have a plus sign or a right-pointing triangle (which means there is hidden, undisplayed information), and the HTTP line has a minus sign or a downpointing triangle (which means that all information about the HTTP message is displayed).

(Note: You should ignore any HTTP GET and response for favicon.ico. If you see a reference to this file, it is your browser automatically asking the server if it (the server) has a small icon file that should be displayed next to the displayed URL in your browser. We'll ignore references to this pesky file in this lab.).

By looking at the information in the HTTP GET and response messages, answer the following questions.





1. Is your browser running HTTP version 1.0, 1.1, or 2? What version of HTTP is the server running?
- 1.1; 1.1
2. What is the IP address of your computer? What is the IP address of the panday.upm.edu.ph server?
- 172.16.124.194; 202.92.148.181
3. What is the status code returned from the server to your browser?
- 200
4. When was the HTML file that you are retrieving last modified at the server in local time (Manila)?
- Mon, 03 Feb 2020 16:37:14 Manila Time
5. How many bytes of content are being returned to your browser?
- 105 bytes

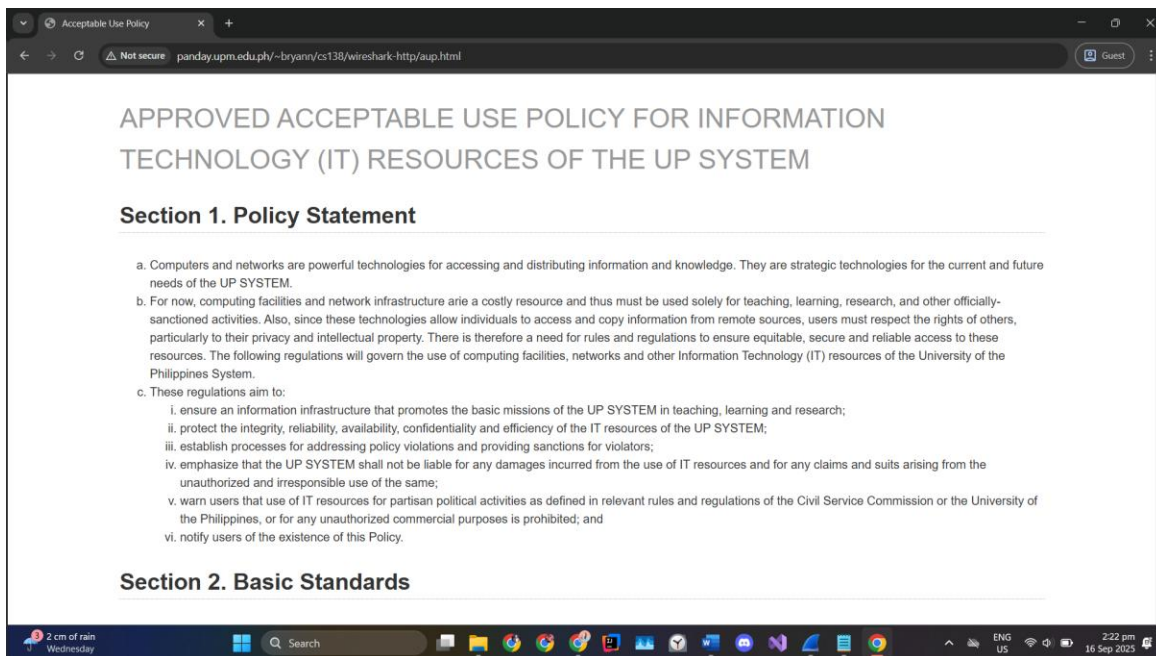
2. Retrieving Long Documents

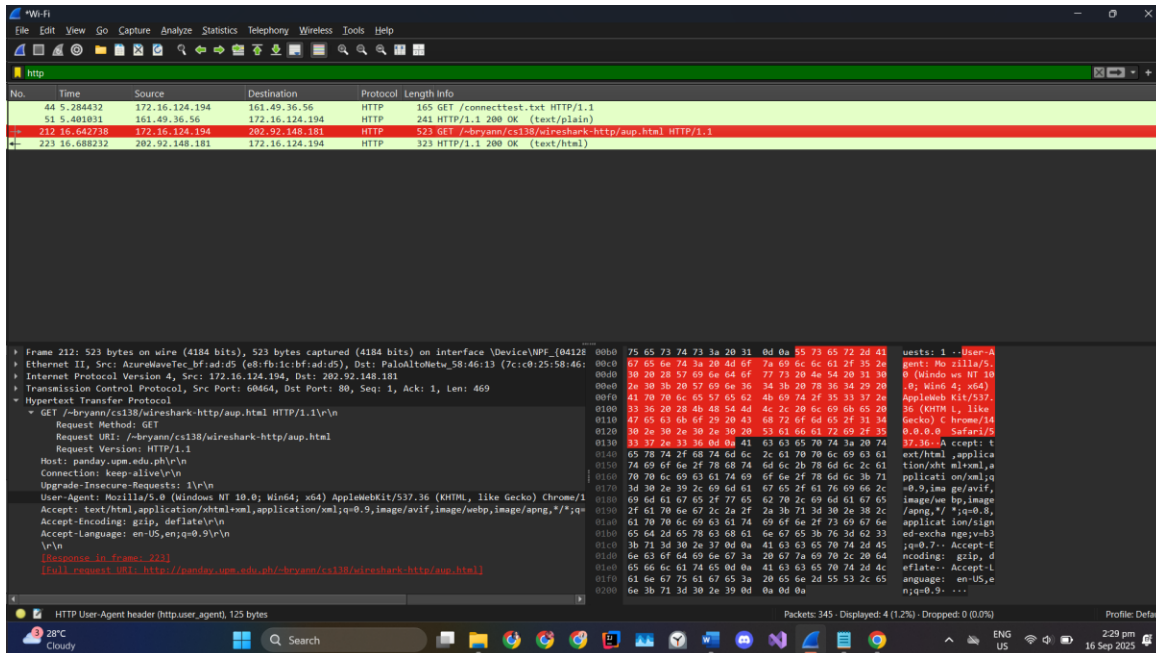
In our examples thus far, the documents retrieved have been simple and short HTML files. Let's next see what happens when we download a long HTML file. Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
<http://panday.upm.edu.ph/~bryann/cs138/wireshark-http/aup.html> Your browser should display the rather lengthy AUP policy of UP.
- Stop Wireshark packet capture, and enter "http" in the display-filter-specification window, so that only captured HTTP messages will be displayed.

In the packet-listing window, you should see your HTTP GET message, followed by a multiple-packet TCP response to your HTTP GET request. Make sure your Wireshark display filter is cleared so that the multi-packet TCP response will be displayed in the packet listing.

This multiple-packet response deserves a bit of explanation. Recall from Section 2.2 (see Figure 2.9 in the text) that the HTTP response message consists of a status line, followed by header lines, followed by a blank line, followed by the entity body. In the case of our HTTP GET, the entity body in the response is the *entire* requested HTML file. In our case here, the HTML file is rather long, and is too large to fit in one TCP packet. The single HTTP response message is thus broken into several pieces by TCP, with each piece being contained within a separate TCP segment. In recent versions of Wireshark, Wireshark indicates each TCP segment as a separate packet, and the fact that the single HTTP response was fragmented across multiple TCP packets is indicated by the “TCP segment of a reassembled PDU” in the Info column of the Wireshark display.





Answer the following questions:

6. How many HTTP GET request messages did your browser send? Which packet number in the trace contains the GET message for the AUP policy?
- 2; 212
7. Which packet number in the trace contains the status code and phrase associated with the response to the HTTP GET request?
- 223
8. What is the status code and phrase in the response?
- 200 OK
9. How many data-containing TCP segments were needed to carry the single HTTP response and the text of the AUP policy?
- 8

3. HTML Documents with Embedded Objects

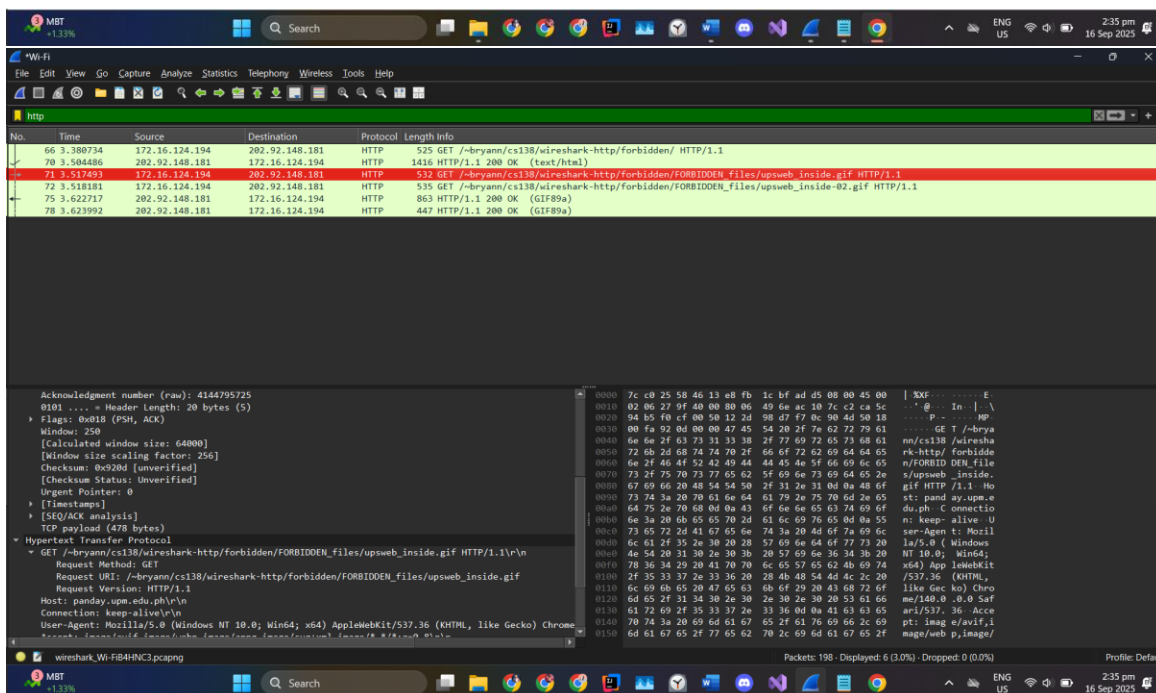
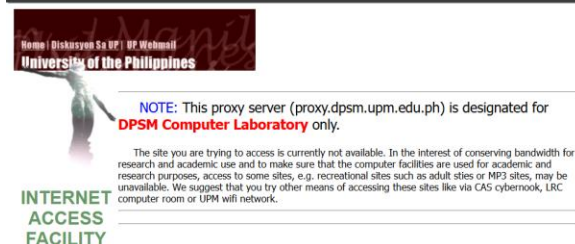
Now that we've seen how Wireshark displays the captured packet traffic for large HTML files, we can look at what happens when your browser downloads a file with embedded objects, i.e., a file that includes other objects (in the example below, image files) that are stored on another server(s).

Do the following:

- Start up your web browser, and make sure your browser's cache is cleared, as discussed above.
- Start up the Wireshark packet sniffer
- Enter the following URL into your browser
<http://panday.upm.edu.ph/~bryann/cs138/wireshark-http/forbidden/> Your browser should display a short HTML file with two images. These two images are

referenced in the base HTML file. That is, the images themselves are not contained in the HTML; instead the URLs for the images are contained in the downloaded HTML file.

- Stop Wireshark packet capture, and enter “http” in the display-filter-specification window, so that only captured HTTP messages will be displayed.



Answer the following questions:

10. How many HTTP GET request messages did your browser send? To which Internet addresses were these GET requests sent?

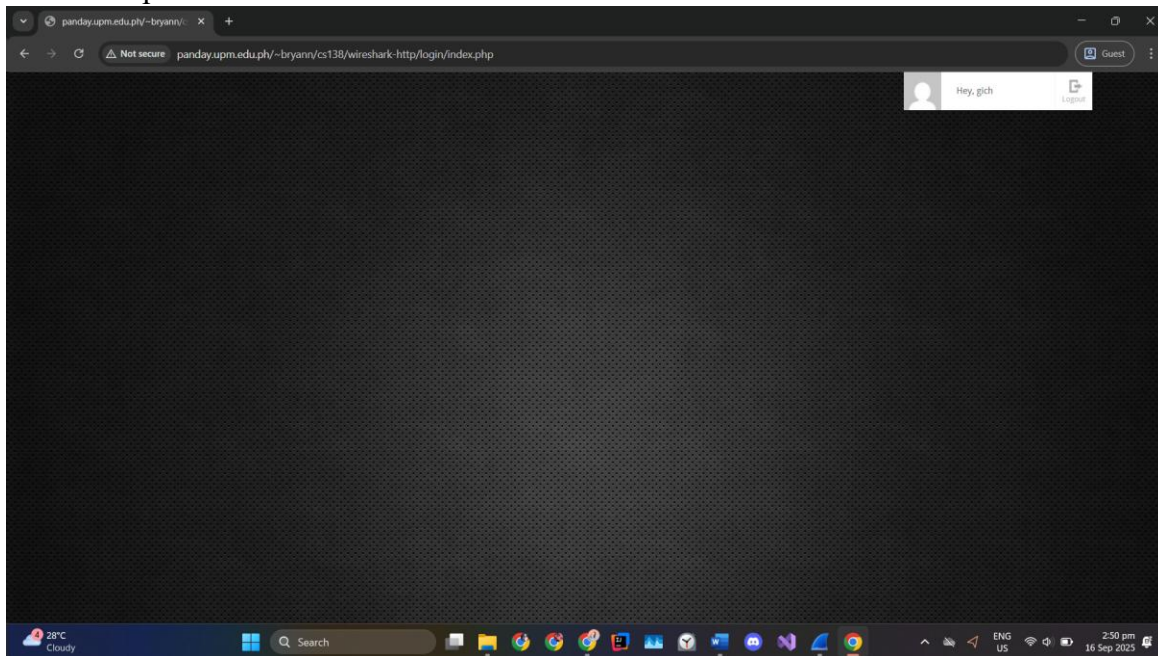
- 3; 202.92.148.181

11. Can you tell whether your browser downloaded the two images serially, or whether they were downloaded from the two web sites in parallel? Explain.
- Parallel; Because the second GET request was sent before the first GIF finished loading and got the status code 200 OK

6 HTTP Authentication

Let us look at the security problem of HTTP by seeing the danger of transmitting confidential information (e.g. username and password) via HTTP instead of via HTTPS.

- Go to <http://panday.upm.edu.ph/~bryann/cs138/wireshark-http/login/>.
- Register for an account. You can use any username, password and email address but **it would be best not to use your usual passwords here**.
- Start your Wireshark.
- Go to <http://panday.upm.edu.ph/~bryann/cs138/wireshark-http/login/> again and use the account that you have used in registration to login.
- Stop Wireshark.



Answer the following questions:

12. You should be able to capture the packet that contain the username and password that you have just entered in the login. Show the packet that contains the username and password.

Capturing on Wi-Fi

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

No.	Time	Source	Destination	Protocol	Length	Info
60	5.846473	192.168.1.2	202.92.148.181	HTTP	510	GET /~bryann/cs138/wireshark-http/login/views/css/style.css HTTP/1.1
66	5.850893	192.168.1.2	202.92.148.181	HTTP	504	GET /~bryann/cs138/wireshark-http/login/views/js/jquery-1.8.1.min.js HTTP/1.1
69	5.851633	192.168.1.2	202.92.148.181	HTTP	502	GET /~bryann/cs138/wireshark-http/login/views/js/jquery.md5.min.js HTTP/1.1
72	5.852172	192.168.1.2	202.92.148.181	HTTP	503	GET /~bryann/cs138/wireshark-http/login/views/js/beautiful_login.js HTTP/1.1
73	5.857300	202.92.148.181	192.168.1.2	HTTP	572	[TCP Previous segment not captured] Continuation
83	5.861469	202.92.148.181	192.168.1.2	HTTP	896	HTTP/1.1 200 OK (text/javascript)
88	5.864438	202.92.148.181	192.168.1.2	HTTP	1301	HTTP/1.1 200 OK (text/javascript)
110	5.916404	192.168.1.2	64.233.187.95	HTTP	578	GET /css?family=Droid+Sans:400,700 HTTP/1.1
125	5.949498	64.233.187.95	192.168.1.2	HTTP	74	HTTP/1.1 200 OK (text/css)
127	5.961042	192.168.1.2	202.92.148.181	HTTP	587	GET /~bryann/cs138/wireshark-http/login/views/img/bg_black_metal_01.jpg HTTP/1.1
128	5.963647	192.168.1.2	202.92.148.181	HTTP	579	GET /~bryann/cs138/wireshark-http/login/views/img/icon_lock.png HTTP/1.1
147	5.971678	202.92.148.181	192.168.1.2	HTTP	1439	HTTP/1.1 200 OK (PNG)
2298	6.788049	202.92.148.181	192.168.1.2	HTTP	1411	HTTP/1.1 200 OK (JPEG JFIF image)
2299	6.796406	192.168.1.2	202.92.148.181	HTTP	513	GET /favicon.ico HTTP/1.1
2300	6.803648	202.92.148.181	192.168.1.2	HTTP	513	HTTP/1.1 404 Not Found (text/html)
2350	10.514088	192.168.1.2	202.92.148.181	HTTP	823	POST /~bryann/cs138/wireshark-http/login/index.php HTTP/1.1 (application/x-www-form-urlencoded)
2357	10.666952	202.92.148.181	192.168.1.2	HTTP	452	HTTP/1.1 200 OK (text/html)
2359	10.689856	192.168.1.2	202.92.148.181	HTTP	626	GET /~bryann/cs138/wireshark-http/login/views/img/icon_logout.png HTTP/1.1
2363	10.702044	202.92.148.181	192.168.1.2	HTTP	316	HTTP/1.1 200 OK (PNG)

Connection: keep-alive\r\n
Content-Length: 46\r\n
Cache-Control: max-age=0\r\n
Origin: http://panday.upm.edu.ph/~bryann/cs138/wireshark-http/login/\r\n
Content-Type: application/x-www-form-urlencoded\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*
Referer: http://panday.upm.edu.ph/~bryann/cs138/wireshark-http/login/\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-US,en;q=0.9\r\n
Cookie: PHPSESSID=qyddibab74mbj63llq3mrqs\r\n
[Response in frame: 2357]
[Full request URL: http://panday.upm.edu.ph/~bryann/cs138/wireshark-http/login/index.php]
File Date: 46 bytes
HTML Form URL Encoded: application/x-www-form-urlencoded
Form item: "user_name" = "gich"
Form item: "user_password" = "gich"
Form item: "login" = "Submit"

01e0 74 6d 6c 2b 78 6d 6c 2c 61 70 70 6c 69 63 61 74 tml+xml, applicat
01f0 69 6f 6e 2f 78 6d 6c 3b 71 3d 30 2e 39 2c 69 6d ion/xml;q=0.9,im
0200 61 62 65 2f 61 76 69 66 2c 69 6d 61 67 65 2f 77 age/xml;q=0.9,im
0210 65 62 70 2c 69 6d 61 67 65 2f 61 70 6e 67 2c 2a ebp,image/apng,*
0220 2f 2a 3b 71 3d 30 2e 38 2c 61 70 70 6c 69 63 61 /*;q=0.8, applica
0230 74 69 6f 6e 2f 73 69 67 6e 65 64 2d 65 78 63 68 tion/sign ned-exch
0240 61 6e 67 65 3b 7c 3d 62 33 3b 71 3d 30 2e 37 0d ange;web;q=0.7
0250 0a 52 65 66 65 72 65 72 3a 20 68 74 74 70 3a 2f Referer : http://
0260 2f 70 61 6e 64 61 79 2e 75 70 6d 2e 65 64 75 2e /panday. upm.edu.
0270 70 68 2f 7e 62 22 79 61 6e 6e 2f 63 73 31 33 38 ph/~brya nn/cs138
0280 2f 77 69 72 65 73 68 61 72 6b 2d 68 74 74 70 2f /wiresha rk-http/
0290 6c 6f 67 69 6e 2f 0d 0a 41 63 63 65 70 74 2d 45 login/- Accept-E
02a0 6e 63 6f 64 69 6e 67 3a 20 67 7a 69 70 2c 20 64 ncoding: gzip, d
02b0 65 6e 6c 61 74 65 0d 0a 41 63 63 65 70 74 2d 4c eflate Accept-L
02c0 61 6e 67 75 61 67 65 3a 20 65 6e 2d 55 53 2c 65 anguage: en-US,e
02d0 6e 3b 71 3d 30 2e 39 0d 0a 43 6f 6f 6b 69 65 3a n;q=0.9. Cookie:
02e0 20 50 48 50 53 45 53 53 49 44 3d 71 39 76 30 64 PHPSESS ID=qydd
02f0 69 6d 61 62 37 34 37 6d 62 6a 62 36 33 6c 6c 71 ibab747m bj63llq
0300 33 6d 72 71 73 0d 0a 0d 75 73 65 72 5f 6e 61 3mrqs = user_na
0310 6d 65 3d 67 69 63 68 26 75 73 65 72 5f 70 61 73 mngicHA user_pas
0320 73 77 6f 72 64 3d 67 69 63 68 26 6f 67 69 6e word=gi chlogin
0330 3d 53 75 62 6d 69 74 =Submit

Text item (text), 14 bytes

Packets: 2631 - Displayed: 21 (0.8%)

Profile: Default

26°C Mostly cloudy

9:55 pm 16 Sep 2023